

A Local Model That Behaves Itself

What the fleet is, how you use it, and what it can actually do

`libr-local-llm` — the next phase

The whole thing in one sentence

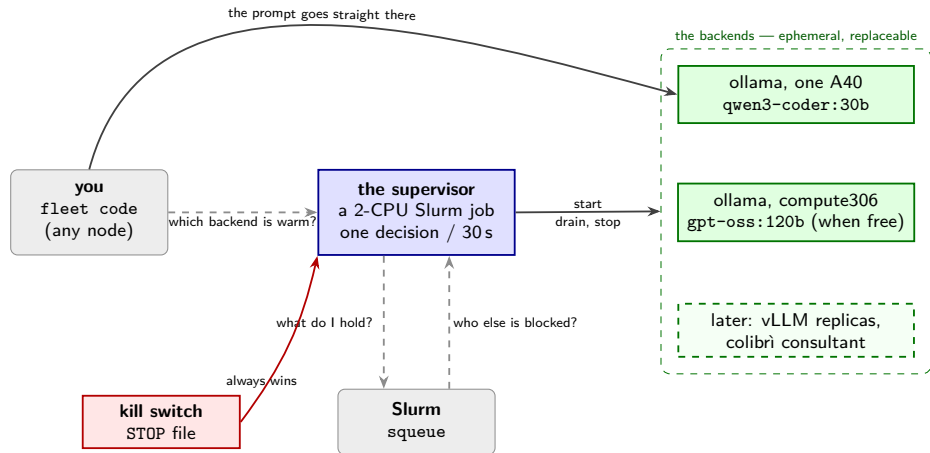
A local coding assistant that is **always there**, **gives cluster resources back** the moment somebody else needs them, and **restarts itself** when it dies.

Three problems it solves, all of which you have today:

Today	Why it hurts	After
You run ollama-up by hand	nothing runs until you notice	it is already up
The server holds a GPU until you kill it	you are the one being emailed	it hands the node back on its own
Walltime ends, the server is gone	9 hours, then silence	a new one takes over, no gap

And one problem it does not solve: it is not Claude. Slide 13 is that conversation, and it is worth reading before the rest.

The system is four boxes



The supervisor remembers nothing. Every 30 seconds it asks Slurm what it holds and who is waiting, compares that to

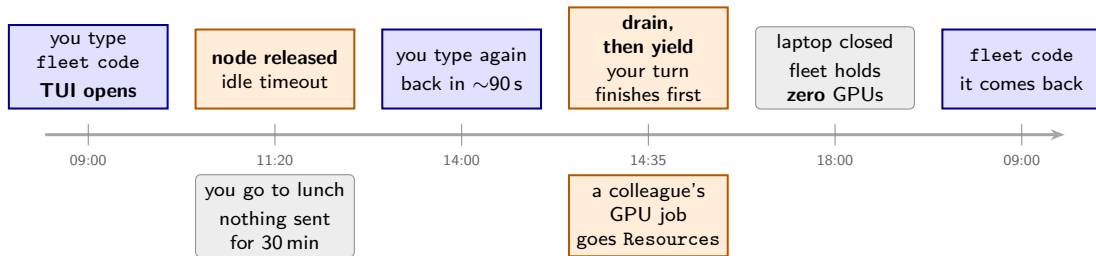
What you actually type

Command	What happens	How long
<code>fleet code</code>	TUI against the local model, from anywhere	seconds, if warm
<code>fleet code "one question"</code>	one-shot answer, no TUI	seconds
<code>fleet status</code>	what is up, what is warm, what it is waiting on	instant
<code>fleet up</code>	start the supervisor if it is not running	~30 s
<code>fleet up --big</code>	also book compute306 for gpt-oss:120b	~5 min cold
<code>fleet down</code>	kill switch: stop everything, stay stopped	<30 s

In practice you type one of these. `fleet code`. It works from the login node, from a VSCode terminal on a compute node, from inside another job. It finds the server, steps onto its node, and opens the TUI you are already used to.

The existing `ollama-up` / `ollama-code` / `ollama-down` keep working, untouched, the whole time. The fleet commands are additive. If the fleet breaks at 2 a.m. you still have the thing that works today.

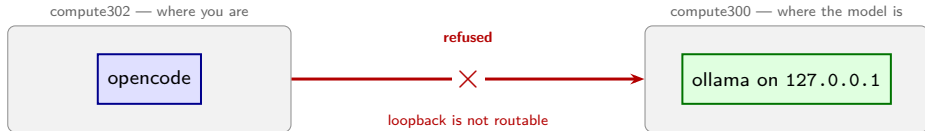
A day in the life



- **Nothing there required you to do anything** except type `fleet` code when you wanted to work.
- **At 14:35 you did not lose your session.** The yield drains first — the answer you were waiting on finishes, *then* the node goes. Session history lives on NFS home, so you reattach by id from wherever you land next.

The one genuinely hard problem

The server binds 127.0.0.1 — loopback, on its own node. That is the control keeping a PHI endpoint off the cluster network. It is also why you cannot just point a client at it.

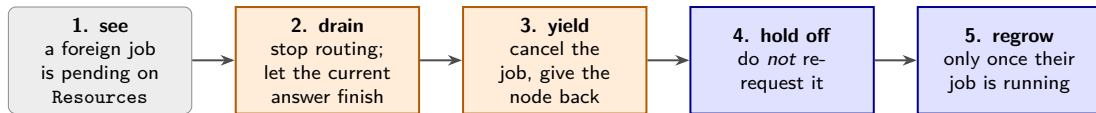


The fix uses machinery this repo already trusts. `srun --overlap` steps into a job on another node and hands you its stdin and stdout. **Slurm itself becomes the wire** — no SSH, no open port, no socket anywhere.



The payoff: the router lives on *your* node, so when the fleet swaps the backend underneath you, the session does not notice. Ships in two stages; stage one just steps you onto the model's node, which works on day one.

Citizenship: what “gives it back” actually means



Step 4 is the one that is easy to skip and ruins everything. Without it: the supervisor releases a node, notices one second later that it is below target, and re-submits — winning the race against the very job it was trying to help. The fleet then looks maximally rude while running code written to be polite.

The rule is not a timer. It is: **do not ask for that resource again until the job that triggered the yield is no longer pending** — with a 15-minute floor underneath it.

Which thing gets given up first

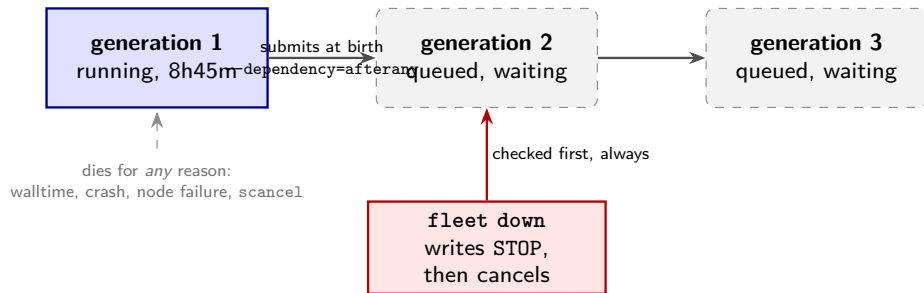
Ordering principle: *surrender the unit holding the most, that costs the least to bring back.*

Rank	Unit	Holds	Costs to restore
1	an idle vLLM replica	1 GPU	a few minutes
2	an idle ollama server	1 GPU	measured 4m46s for 65 GB
3	a compute306 booking	4 GPUs	minutes; biggest single win
4	a <i>busy</i> vLLM replica	1 GPU	minutes + the in-flight work
5	the colibrì backend	1 GPU + ~500 GB RAM	projected ~31 minutes

Notice what the ordering does to colibrì without anybody writing a special case for it: it holds the *least* GPU and takes *longest* to come back, so it lands at the bottom naturally.

Why restore cost dominates. There is no local scratch disk on these nodes, so every cold model load is an NFS read at **measured** ~230 MB/s. A fleet that thrashes is worse than a fleet that is simply smaller. Hysteresis is not tuning — it is correctness.

Restarting itself, and the switch that stops it



Each generation submits its successor the moment it starts, with a dependency that fires on *any* termination — walltime, a crash, a node vanishing. A pending job costs nothing, so the queued successor is free insurance.

The kill switch is a file, not a signal. `fleet down` writes STOP *before* it cancels anything, so the successor starts, reads it, and exits in two seconds. It works when the supervisor is wedged and when its node is gone — a kill switch that depends on the thing it kills is not one.

How fast, honestly

Tier	Model	VRAM	Speed	Where it fits
fast	qwen3-coder:30b	18.6 GB	projected 40–60 tok/s	any one A40 — 6 nodes
big	gpt-oss:120b	70 GB	measured 34.1 tok/s	4×A40 — compute306 only
huge	GLM-5.2 744B int4	(RAM)	projected high single digits	1 A40 + ~500 GB RAM

Warm is the number that matters.

34.1 tok/s reads comfortably. 40–60 *works* comfortably — an agent making tool calls needs speed more than brilliance.

Cold is the number that hurts.

measured 4m46s to page gpt-oss:120b off the share before its first token. Every yield decision is arithmetic about that number.

The structural rule that falls out: the *fast* model drives the agent loop. The big models are **consultants** — asked one hard question, given a few hundred tokens to answer it. Thirty seconds of consultation inside a five-minute task is invisible. Driving the whole loop at 8 tok/s is unusable.

What this is not

It is not Claude, and no amount of plumbing makes it Claude.

The gap is the **weights**, not the serving. Three specific things stay true after everything above is built:

- **The ceiling is gpt-oss:120b**, a good open 120B coder — and it needs the cluster's only four-GPU node, which is exactly the resource the design says never to camp on.
- **The 744B model is not automatically better.** Its int4 container measures **−8.2 percentage points** of quantization damage, concentrated on the hardest questions. A 744B model at int4 beating a well-served 120B at 8-bit is a *hypothesis*, and nobody has run the comparison on our tasks.
- **No routing layer fixes this.** No arrangement of three engines is smarter than the best model among them. The fleet buys coverage, uptime and politeness — not intelligence.

So why build it? One reason, and it is enough:

A frontier-scale model that **may read PHI**. No hosted service will ever be allowed to do that here. That is the capability, and it is not available any other way.

What it costs us

The hardware is bought and powered. The real bill is in two other currencies.

Currency	Amount	Who pays
other people's queue time	bounded by caps + yielding	colleagues
our own fair-share	~500,000 usage units at steady state	our own research jobs

The second one is the surprise. A supervisor running around the clock holds Slurm's minimum billable unit — two CPUs, always, because cores here carry two threads and nothing smaller can be allocated. At the cluster's two-day decay that reaches a steady-state usage contribution **roughly the size of everything this account currently runs**. Fair-share is the heaviest term in the priority formula, so a fleet that never sleeps makes the PSYCH-ASR and TRD-EHR jobs it exists to serve schedule *worse*.

The fix is boring and it works: when the fleet has held no GPUs for two hours, the supervisor exits too. Typing `fleet` code brings it back in about thirty seconds. A supervisor watching an empty fleet is indistinguishable, from your seat, from no supervisor at all.

Three cluster facts nobody had checked

Found while planning this, on the live scheduler. Each one changes something.

- ❶ **The c3 partition can silently freeze our server.** `c3_short` outranks `c3` on the same six nodes, and the configured response is *suspend* — `SIGSTOP`, not `requeue`. A suspended server keeps its VRAM (so the preemption helps nobody) and stops mid-generation with the socket open, so the client gets no error, just silence. **Consequence:** the fleet never uses `c3`. The README currently recommends it for long-lived servers; that advice is a trap.
- ❷ **Asking for memory silently buys CPUs.** Memory is capped at 12 GB per CPU here, so `--mem=64G --cpus-per-task=4` does not get 4 CPUs — it gets 6. Our careful CPU-rightsizing argument was being undone by the memory line the whole time.
- ❸ **Almost nothing on this cluster is actually blocked.** Of the 27 pending jobs visible the day this was written, **all 27** were waiting on a start time they had asked for, not on resources. A naive “somebody is waiting, give a node back” rule would surrender the entire fleet, continuously, to jobs it was not blocking. **Consequence:** the yield filter is the load-bearing part, and the yield path has to be tested on purpose because it will rarely fire on its own.

Build order — each step is useful before the next exists

Phase	What lands	What you get	Needs a GPU?
P0	8 verification tests	the facts on slide 12, confirmed	no
P1	supervisor, chain, kill switch	something that never dies	no
P2	backends, caps, idle release	it starts and stops servers for you	yes
P3	the yield ladder	it is a good citizen	yes
P4a	fleet code	you stop typing ollama-up	yes
P4b	the relay	sessions survive a backend swap	yes
P5	vLLM, colibri	throughput, and the 744B consultant	yes

P4a is the moment it becomes worth having. Everything before it is machinery; that is the phase where the daily command changes. P5 is a different project wearing the same name — do not start it until the fleet has run unattended for a week.

P0 is a day. It goes first because any one of its eight tests can invalidate a design decision above it, and finding that out after P3 is expensive. One of the eight submits jobs to a shared queue and gets asked about first.

Five decisions that are yours, not mine

Question	Proposed	Cost if changed
Does any backend ever bind a network interface, instead of loopback?	No. Use the relay.	Yes weakens the strongest PHI control we have
How many of the six single-GPU nodes may the fleet hold?	2	more is ruder, fewer is slower
May it book compute306 on its own?	No — only fleet up --big	it is the only 4-GPU node on the cluster
Permanent supervisor, or exit when idle?	Exit after 2 idle hours	permanent roughly doubles our recent usage
Which model is the default fast tier?	qwen3-coder:30b	a bigger 8-bit model also fits 46 GB; nobody has measured the trade

These are recorded as open in `FLEET-BUILD.md` §11. Answer them once, with a date, and the implementation stops guessing.

What “done” looks like

You type `fleet` code on any node and get a TUI against a local model in under a minute.

You close the laptop. Overnight the fleet releases everything it holds and the supervisor exits.

In the morning `fleet` code brings it back.

If a colleague's GPU job goes pending at 2 p.m., the fleet finishes your answer, gives the node back, and does not take it again until their job is running — and the log says so, with their job id, when anybody asks.

Nothing in that paragraph requires an administrator, a daemon on a login node, or a person watching.

That is the deliverable. It is achievable. It is not Claude.